

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Nástroj pro automatickou kontrolu programátorských
úkolů



2026

Vedoucí práce:
Mgr. Petr Osička, Ph.D.

My Linh Duongová

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor: My Linh Duongová
Název práce: Nástroj pro automatickou kontrolu programátorských úkolů
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Informatika, Specializace: Programování a vývoj software
Vedoucí práce: Mgr. Petr Osička, Ph.D.
Počet stran: 30
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: My Linh Duongová
Title: Automated checker for programming assignments
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Computer Science, Specialization: Programming and Software Development
Supervisor: Mgr. Petr Osička, Ph.D.
Page count: 30
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Při náboru na stážistické pozice ve firmách se programátorské úkoly uchazečů často kontrolují manuálně, což je při vysokém počtu zájemců časově velmi náročné. Práce na tento problém reaguje vývojem a implementací platformy InQuest pro firmu Edhouse, která celý proces validace automatizuje. Systém si po zadání odkazu na GitHub repozitář kód bezpečně stáhne a spustí ho v izolovaném kontejneru přes Podman nebo Docker. Zde ověří strukturu složek a zachytí výstupy z kompilace i samotného běhu programu. Řídící aplikace je napsaná v C# (.NET) a pro spolehlivou komunikaci s testovacími kontejnery využívá frontu zpráv RabbitMQ, přičemž výsledné reporty jsou přes API dostupné pro potřeby firmy.

Synopsis

When recruiting for internship positions in companies, the programming tasks of applicants are often checked manually, which is very time-consuming when there is a large number of applicants. This work responds to this problem by developing and implementing the InQuest platform for the company Edhouse, which automates the entire validation process. After entering a link to the GitHub repository, the system safely downloads the code and runs it in an isolated container via Podman or Docker. Here, it verifies the folder structure and captures the outputs from compilation and the program run itself. The control application is written in C# (.NET) and uses the RabbitMQ message queue for reliable communication with test containers, with the resulting reports available for the company's needs via API.

Klíčová slova: automatizované testování, Podman, kontejnerizace, RabbitMQ, GitHub API

Keywords: automated testing, Podman, containerization, RabbitMQ, GitHub API

TODO: Děkuji, děkuji, děkuji.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	8
1.1	TO EXPAND: Motivace	8
1.2	Cíl práce	8
2	Použité technologie	9
2.1	C# .NET	9
2.1.1	ASP.NET Core Web API	9
2.1.2	Entity Framework Core	9
2.1.3	Swagger (OpenAPI)	10
2.1.4	AutoMapper	10
2.1.5	Serilog	10
2.2	REST API	10
2.3	Podman	10
2.4	RabbitMQ	11
2.5	GitLab CI/CD	12
2.5.1	Formát YAML	12
2.5.2	BASH skriptování	12
2.6	GitHub API	12
3	Architektura a návrh systému	13
3.1	Clean Architecture	13
3.2	Backend	14
3.2.1	Prezentační vrstva	14
3.2.2	DTO	14
3.2.3	Doména	14
3.2.4	Aplikace	15
3.2.5	Infrastruktura	15
3.3	Runners	15
4	Automatizované sestavení a nasazování	17
5	Implementace	18
5.1	TO DO: Workflow systému	18
5.2	Průběh zpracování požadavku	18
5.3	TO DO: Příjetí požadavku	19
5.4	TO DO: Izolovaná exekuční prostředí (Runnery)	19
5.4.1	TO DO: Spuštění izolovaného prostředí	19
5.4.2	BASH skripty a úprava prostředí	19
5.5	TO DO: Zpracování výsledku	20
5.6	TO DO: Ukázky použití	20
6	TO FINISH: Budoucí rozšíření	21

7	TO DELETE: Sazba částí dokumentu	22
7.1	Sazba úvodní strany či obsahu	22
7.2	Závěry	22
7.3	Sazba literatury	22
7.4	Drobná makra	22
7.5	Sazba rejstříku	23
7.6	Sazba zdrojových kódů	23
	Závěr	26
	Conclusions	27
A	TODO: Obsah elektronických dat	28
	Literatura	29

Seznam tabulek

1	Odstavce v tabulkách	24
---	--------------------------------	----

1 Úvod

1.1 TO EXPAND: Motivace

Firma Edhouse, zabývající se vývojem software a hardware, přijímá každým rokem nové stážisty. Kvůli velkému zájmu ale vyhodnocování vstupních programátorských úkolů začalo být neudržitelné. Nejen že museli mentoři úkoly manuálně stahovat a testovat, museli kontrolovat úkoly v různých jazycích. Tento proces zdržoval náborové řízení a bral hodně času programátorům od jejich dedikované práci. Motivací aplikace InQuest je tedy proces kontroly stážistických úkolů zautomatizovat, aby se mentoři mohli zabývat až posudkem kvality kódu.

1.2 Cíl práce

TO THINK: analyzovat existující řešení a nechat cíl práce jako jeho závěr?

Cílem bakalářské práce je navrhnout a vyvinout systém pro kontrolu programátorských úkolů. Výsledný systém musí být schopen autonomně stáhnout kód z GitHub repozitáře, detekovat programovací jazyk a v izolovaném kontejnerovém prostředí bezpečně ověřit strukturu projektu, úspěšnost kompilace a funkčnost běhu programu. Technickým cílem je postavit robustní backend v C# (.NET) s asynchronní architekturou využívající RabbitMQ.

Hlavní cíle tedy jsou:

- Analýza požadavků: Definovat potřeby firmy ohledně struktury testovacích úkolů a formátu výsledných reportů.
- Izolace a validace: Navrhnout kontejnerové prostředí, které zajistí bezpečné sestavení, spuštění a analýzu cizího kódu bez rizika pro firemní infrastrukturu.
- Asynchronní architektura: Propojit řídicí aplikaci v C# (.NET) s exekucí kontejnery pomocí message brokera RabbitMQ.
- Nasazení a integrace: Připravit API rozhraní pro snadné poskytování výsledných reportů.

2 Použité technologie

Při návrhu systému bylo potřeba zvolit technologie, které umožní bezpečné spouštění cizího zdrojového kódu, asynchronní komunikaci mezi jednotlivými komponentami a snadnou rozšiřitelnost o kontrole dalších programovacích jazyků. Tato kapitola představuje použité technologie, důvody jejich výběru a jejich roli v navrženém systému.

2.1 C# | .NET

Pro implementaci backendu byl vybrán jazyk C# na platformě .NET 8 [1] pro svou robustnost, objektově orientovaný přístup a vhodnost pro vrstvenou a modulární architekturu. Tato technologie byla zvolena hlavně kvůli rozsáhlé podpoře, vysokému výkonu a předpokladům pro kontejnerizaci. Významnější konkrétní použité knihovny jsou více popsány v následujících podsekcích.

2.1.1 ASP.NET Core Web API

ASP.NET Core Web API [2] je framework určený pro vývoj webových služeb typu REST na platformě .NET. Umožňuje vytvářet lehká, rychlá a škálovatelná rozhraní HTTP, prostřednictvím kterých mohou mezi sebou komunikovat jednotlivé aplikace nezávisle na použité technologii.

Framework plně využívá pokročilých vlastností prostředí ASP.NET Core, mezi které patří vestavěné vstřikování závislostí (*dependency injection*), verzování, asynchronní programování (*async/await*) a efektivní směrování požadavků (*routing*). Dále nabízí nativní middleware pro správu životního cyklu požadavků, bezpečnost, chybové stavy i snadnou integraci se šablonou OpenAPI (Swagger, viz podkapitolu 2.1.3) pro automatickou dokumentaci API.

2.1.2 Entity Framework Core

Pro přístup k datům a řízení databázové vrstvy byl zvolen framework Entity Framework Core (EF Core) [4]. Jedná se o moderní objektově-relační mapovač (ORM), který slouží jako abstraktní mezivrstva mezi objektově orientovaným kódem v C# a relační databází. EF Core eliminuje potřebu ručního psaní SQL dotazů a umožňuje manipulovat s daty prostřednictvím C# objektů. Pro přístup k databázi využívá třídu `DbContext` a podporuje tvorbu dotazů pomocí rozhraní LINQ (*Language Integrated Query*), které je během běhu aplikace automaticky překládáno do SQL příkazů odpovídajících použitému databázovému stroji.

V rámci vývoje byl aplikován přístup *Code First*. Tato metodika spočívá v tom, že je databázové schéma definováno přímo pomocí doménových tříd v kódu. Změny v modelu jsou následně spravovány a propagovány do databáze pomocí systému migrací [3], což zajišťuje konzistenci databázové struktury a její snadné verzování v řízení zdrojového kódu společně s aplikací.

2.1.3 Swagger (OpenAPI)

Swagger [5] představuje nástroj určený pro automatické generování dokumentace REST API (viz podkapitolu 2.2) podle specifikace OpenAPI. Na základě definovaných kontrolerů a datových modelů vytváří přehlednou dokumentaci jednotlivých endpointů, jejich vstupních parametrů, návratových hodnot i možných chybových stavů.

Při vývoji bylo využíváno jeho webové rozhraní Swagger UI, které umožnilo jednotlivé endpointy přímo testovat bez nutnosti využití externích aplikací, například Postmanu. Takto se mohla snadno ověřit správnost implementace API a zároveň byla získána přehledná dokumentace určena pro další vývojáře.

2.1.4 AutoMapper

AutoMapper [6] je knihovna určená pro automatické mapování objektů mezi různými datovými modely. Nejčastěji se využívá při převodu databázových entit na datové přenosové objekty (angl. Data Transfer Object, dále jen DTO) a naopak.

Bez využití mapovací knihovny je nutné každou vlastnost objektu kopírovat ručně, což vede ke vzniku značného množství redundantního kódu. AutoMapper tento proces pro vývoj automatizuje na základě předem definovaných mapovacích profilů, ve kterých jsou stanovena pravidla převodu mezi jednotlivými třídami.

2.1.5 Serilog

Pro strukturované zaznamenávání diagnostických událostí během vývoje byla využita knihovna Serilog [7]. Knihovna je nakonfigurována pro výstup do konzole (*Console Sink*), kde zajišťuje přehledné a barevně odlišené formátování logů včetně zachování datových struktur požadavků, což usnadňuje ladění a sledování běhu API v průběhu vývoje. Umožňuje zaznamenávat různé úrovně závažnosti událostí, jako jsou informace, varování nebo chyby.

2.2 REST API

REST (*Representational State Transfer*) [8] je architektonický styl pro návrh distribuovaných webových služeb. Komunikace probíhá prostřednictvím protokolu HTTP a využívá standardních metod, jako jsou GET, POST, PUT nebo DELETE.

Jednou z hlavních vlastností REST API je bezstavovost (stateless), kdy každý požadavek obsahuje všechny informace potřebné pro jeho zpracování. Díky tomu lze jednotlivé požadavky zpracovávat nezávisle na sobě, což usnadňuje škálování aplikace.

2.3 Podman

Vzhledem k tomu, že systém pracuje s neověřenými zdrojovými kódy od externích uchazečů, bylo nutné zajistit bezpečné spouštění v izolovaném prostředí.

Proto byla zvolena kontejnerizace, která odděluje běh testované aplikace od hostitelského operačního systému.

Podman [9] (s alternativní možností Dockeru) je open-source nástroj pro správu kontejnerů kompatibilní se standardem OCI (*Open Container Initiative*). Poskytuje téměř totožné příkazové rozhraní jako Docker, díky čemuž lze většinu existujících `Dockerfile` i příkazů používat beze změny.

Na rozdíl od Dockeru využívá Podman tzv. *daemonless* architekturu, tedy nevyžaduje centrální službu běžící na pozadí. Kontejnery vytváří přímo jako procesy operačního systému a podporuje také *rootless* režim, ve kterém mohou běžet bez administrátorských oprávnění. To představuje vhodnější řešení pro spouštění nedůvěryhodného kódu, protože případný únik z kontejneru nezíská oprávnění správce hostitelského systému. [10]

Pro bezpečné předávání citlivých informací využívá Podman funkci *Podman Secrets* [11]. To umožňuje ukládat tajné hodnoty, například přístupové tokeny nebo hesla, mimo samotný kontejnerový obraz. Secret je při spuštění kontejneru připojen jako dočasný soubor, který je dostupný pouze běžícímu kontejneru a není součástí výsledného obrazu ani proměnných prostředí. Tím je minimalizováno riziko neúmyslného zveřejnění citlivých údajů.

Další výhodou tohoto nástroje je řízení kontejnerů přes jeho nativní službu REST API (Libpod API) [12]. Prostřednictvím HTTP rozhraní zpřístupňuje veškerou správu kontejnerů. Takto může aplikace automaticky vytvářet nové kontejnery, předávat jim vstupní proměnné, sledovat jejich stav a po dokončení práce zajišťovat jejich smazání.

Při návrhu byly zvažovány nástroje Docker i Podman. Vzhledem k tomu, že aplikace spouští nedůvěryhodný zdrojový kód uchazečů, byl zvolen Podman hlavně kvůli, již zmíněným, podporám **rootless režimu** a **daemonless architektuře**. Tyto vlastnosti umožňují bezpečnější provoz bez nutnosti spouštět centrální službu s administrátorskými oprávněními.

2.4 RabbitMQ

Zprávový broker (angl. message broker) RabbitMQ [13] zajišťuje asynchronní komunikaci a spolehlivé předávání zpráv mezi hlavní aplikací a izolovanými výpočetními prostředími. V architektuře systému plní roli prostředníka při řízení toku úloh. Izolované kontejnery vyhrazené pro spouštění externího kódu publikují po dokončení své úlohy zprávu s výsledkem do fronty, ze které ji hlavní backendová aplikace asynchronně konzumuje a zpracovává. Tento přístup vytváří volnou vazbu (*loose coupling*) mezi jednotlivými komponentami, zabráňuje blokování hlavní aplikace během náročných výpočtů a garantuje, že se zprávy s výsledky neztratí ani při náhlém zvýšení zátěže.

2.5 GitLab CI/CD

GitLab CI/CD [14] je integrovaný nástroj pro automatizaci vývojového cyklu aplikací (Continuous Integration / Continuous Delivery). Umožňuje automaticky spouštět definované úlohy při každém odeslání kódu do vzdáleného repozitáře. Samotné spouštění testů, sestavování nebo nasazování probíhá na oddělených serverech či kontejnerech (tzv. GitLab Runners). Hlavní výhodou je eliminace manuálních kroků při nasazování a okamžitá zpětná vazba o stavu sestavení po každé změně.

2.5.1 Formát YAML

YAML (*YAML Ain't Markup Language*) [15] je datový a konfigurační formát zaměřený na dobrou čitelnost. Místo složité syntaxe se závorkami nebo párovými značkami (jako u JSON či XML) využívá k vymezení struktury a odsazování řádků mezery. Je standardem pro psaní konfiguračních souborů u kontejnerových nástrojů a CI/CD platform.

2.5.2 BASH skriptování

BASH (*Bourne Again Shell*) [16] je příkazový interpret a skriptovací jazyk běžný v prostředí Linuxu. Ve spojení se standardními nástroji pro zpracování textu (jako `sed` nebo `xmllstarlet`) umožňuje psát skripty pro automatizaci systémových kroků, manipulaci se soubory nebo úpravu textových dat přímo v příkazovém řádku.

2.6 GitHub API

GitHub API [17] představuje RESTové rozhraní umožňující přístup k informacím uloženým na platformě GitHub. Přes HTTP požadavky lze získávat metadata o repozitářích, uživateli, větvích nebo jednotlivých souborech.

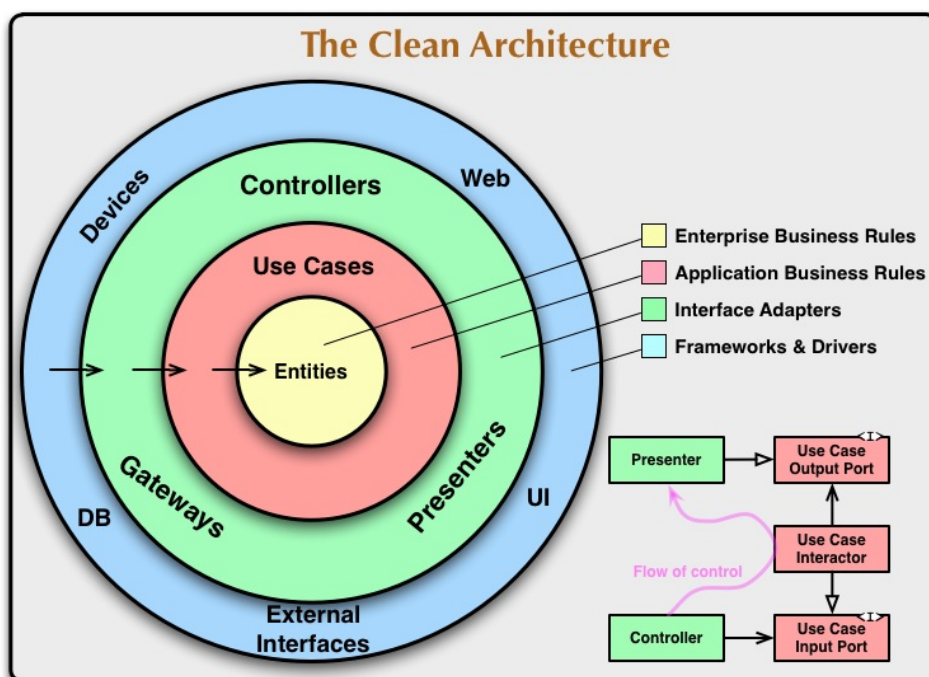
Komunikace s GitHub API probíhá pomocí osobního přístupového tokenu (Personal Access Token), který umožňuje přístup i k neveřejným repozitářům a současně omezuje riziko překročení limitů pro anonymní požadavky.

3 Architektura a návrh systému

Na základě požadavků popsaných v předchozí kapitole bylo navrženo řešení umožňující automatizované vyhodnocování programátorských úloh. Systém musí zajistit bezpečné spuštění nedůvěryhodného kódu, zpracování více programovacích jazyků, komunikaci s externími službami a ukládání výsledků vyhodnocení. Současně bylo cílem navrhnout architekturu umožňující jednoduché rozšíření o nové programovací jazyky nebo další validační pravidla bez zásahů do hlavní aplikační logiky.

Z těchto důvodů byla pro backend zvolena architektura založená na principech Clean Architecture, která odděluje obchodní logiku od konkrétních technologií a externích závislostí.

3.1 Clean Architecture



Obrázek 1: Čistá architektura

Česky "Čistá architektura" [18] je návrhový vzor, který definuje strukturu softwarového systému tak, aby byla nezávislá na frameworku, UI, databázi a externích agentech. Díky oddělení byznysové logiky a technických detailů, zajišťuje snadnou údržbu, testovatelnost a abstrakci mezi vrstvami.

Podle striktního pravidla závislostí (angl. The Dependency Rule) je směr vrstev definován do středu. Vnitřní vrstvy nemají povědomí o tom, co se děje ve vnějších vrstvách.

Vrstvy se dělí na následovné:

- **Doménová** (Domain/Entity): nachází se v samotném jádře architektury. Při externích změnách zde není očekávaná změna. Nemá žádné externí závislosti a obsahuje jen objekty, rozhraní a funkce.
- **Aplikační** (Application/Use Cases): definuje případy užití aplikace. Zahrnuje služby, dotazy a validátory.
- **Infrastrukturní** (Infrastructure): vrstva, která obsahuje databáze nebo externí komponenty.
- **Prezentační** (Presentation / API): vystavuje koncové body pro komunikaci přes HTTP. Jejím úkolem je přijímat požadavky, předávat je aplikační vrstvě ke zpracování a zobrazovat výsledky.

3.2 Backend

Backend aplikace je rozdělen do několika projektů odpovídajících vrstvám Clean Architecture. Jednotlivé vrstvy mají jasně vymezené odpovědnosti a komunikují prostřednictvím definovaných rozhraní. Díky tomu lze měnit implementaci infrastruktury nebo externích služeb bez zásahu do aplikační logiky.

3.2.1 Prezentační vrstva

Ve vrstvě (`Edhouse.inQuest`) se inicializuje Swagger, jako interaktivní RESTová dokumentace a registrují se všechny služby pro Dependency Injection. Pro logování chování aplikace se využívá framework Serilog.

Projekt obsahuje složky `Controllers`, jejichž úkolem je řízení kontrolerů a jejich koncových bodů, a `Filters`, které zachycují výjimky. A nakonec `Mapper`, kde se převádějí objekty aplikace na DTO, nebo databázové entity naopak.

3.2.2 DTO

Vrstva datových přenosových objektů představuje unifikovaný datový kontrakt pro celý systém. DTO jsou reprezentovány jako třídy bez byznys logiky, které přenášejí data mezi jednotlivými vrstvami a externími systémy.

3.2.3 Doména

Doménová vrstva v projektu obsahuje byznys model nezávislý na jakýchkoli frameworkcích, databázích či externích knihovnách. Zatímco ostatní vrstvy, kromě DTO vrstvy, mohou záviset na této, doménová nezávisí na jiné. V tomto specifickém systému existují projekty modelů pro abstrakce (rozhraní služeb), výjimky, nastavení a tak.

3.2.4 Aplikace

Aplikační vrstva obsahuje orchestraci datových toků a validační logiku procesů. Obsahuje tři složky s případy užití.

- **/AssignmentEvaluation** slouží k parsování poskytnutého URL ke GitHub repozitáři, zjistí se programovací jazyk pomocí GitHub API.
- **/Services** je popis poskytovaných služeb.
 - **Assignment**: spravuje životní cyklus assignmentu a stav ukládá do databáze.
 - **Result**: zpracovává výstup po vyhodnocení, zparsuje stdout na build/run výstup, nastavuje příznaky úspěchu a ukládá výsledek do databáze.
 - **Value**: kontroluje hodnoty výstupu. Pro každé zadání by se měl tento modul měnit.
- **/Validator** je složka pro služby, které se týkají validace.

3.2.5 Infrastruktura

Infrastrukturní vrstva zajišťuje komunikaci aplikace s externími komponentami a databází. Mezi její hlavní součástí patří *Entity Framework Core* pro správu databáze. Dále se zde nachází *PodmanManager*, který komunikuje se službou Podman přes HTTP REST rozhraní. Ten řídí spouštění a mazání kontejnerů. Infrastruktura obsahuje napojení na message broker *RabbitMQ*, jenž zajišťuje asynchronní příjem zpráv z kontejnerů a jejich předání na zpracování. A nakonec *GitHub API* pro zjištění jestli GitHub repozitář existuje a pro získání jeho programátorského jazyka.

3.3 Runners

Protože aplikace podporuje více programovacích jazyků, bylo nutné navrhnout jednotný způsob jejich zpracování. Každý jazyk využívá vlastního exekučního běžce (runner), jehož úkolem je ověřit strukturu projektu, provést sestavení aplikace a spustit ji v izolovaném prostředí. Všechny runnery sdílí společnou část implementace a liší se pouze skriptem obsahujícím příkazy specifické pro daný programovací jazyk.

Složka obsahuje následující vybraná skripta:

- **checkVariables.sh** je skript, který kontroluje přítomnost potřebných proměnných prostředí (environment variables) při startu kontejneru. Prvně zjistí, jestli byl přiložen URL ke GitHub repozitáři a poté zjistí, jestli má kontejner přístup k cestě pro access token. Ten je nezbytný, jestli je repozitář soukromý. Ale i kdyby byl veřejný, je potřeba nastavit access token s Podman secrets.

- **generateValidationResult.sh** a **reportResult.sh** se starají o vygenerování a zformátování DTO s informací ohledně průběhu projektu uchazeče a jeho následné odeslání pomocí služby RabbitMQ do zprávové fronty.
- **validate.sh** obsahuje logiku pro stahování GitHub repozitáře, vyjmutí informací, spuštění konkrétní implementace jazykového skriptu `start.sh` s timeoutem a vypsáním následných zpráv po ukončení běhu projektu.
- **run.sh** je hlavní řídicí skript, který orchestruje a spouští ostatní skripty.

Konkrétní implementace pro jednotlivé programovací jazyky se nachází pod složkami s jejich zkratkou. Využívá se výše popsany základ se svou vlastní logikou a balí je do výsledného kontejnerového obrazu. Každá složka obsahuje:

- **runnerScript-<language_extension>.sh** reprezentuje konkrétní implementaci daného jazyka. Kontroluje přítomnost povinných souborů v projektu a volá specifické příkazy technologie ke kompilaci a spuštění běhu.
- **Dockerfile** definuje předpis pro sestavení obrazu. Přibalí se zde sdílené skripty, stáhnou se dodatečné systémové příkazy podle skriptu `../installPrerequisites.sh`, přejmenuje se skript daného jazyka na `start.sh` a nastaví se ENTRYPOINT na skript `run.sh`.

Díky této architektuře je proces přidání nového jazyka velice snadný. V dokumentaci `testRunnerDocumentation.md`, nacházející se v podsložce `./documentations`, je detailní popis jak vytvořit takovou složku.

4 Automatizované sestavení a nasazování

Jalikož aplikace využívá větší množství kontejnerových obrazů a podporuje postupné rozšiřování o nové programovací jazyky, bylo potřeba automatizovat proces sestavení a nasazení. Z tohoto důvodu byla vytvořena CI/CD pipeline v prostředí GitLab. Konfigurace je definována v souboru **gitlab-ci.yml**. Pipeline zjišťuje automatizované verzování, kompilaci backendu, přípravu Docker obrazů a dynamické sestavení izolovaných testovacích runnerů.

Pipeline je rozdělena do šesti následujících fází, které nesou v názvu koncovku `__job`:

- **update**: Tato fáze zajišťuje, že každé sestavení ponese unikátní číslo verze odpovídající historii v GitLabu. Skript nejdříve vyparsuje základní verzi ze souboru `Directory.Build.props`. K této verzi se připojí identifikátor běžící pipeline. Výsledné číslo verze (např. 1.0.67) je pomocí **xmlstarlet** zpětně zapsáno do atributů `AssemblyVersion` a `PackageVersion` a následně předáno jako artefakt souboru `version.env` pro další kroky.
- **build a deploy**: Zde se na základě definovaného Dockerfile sestaví obrazy aplikace s tagem unikátního čísla verze získaného z předchozího kroku a s tagem `latest`. Tyto obrazy jsou poté ve fázi **deploy** nahrány do registru, odkud si je server může stáhnout pro produkční nasazení.
- **build-test-runners a deploy-test-runners**: Stejně jako fáze **build**, tato také sestavuje obrazy pro jednotlivá prostředí různých programovacích jazyků. A díky tomu, že aplikace InQuest podporuje modularitu a přidávání nových programovacích jazyků pouhým přidáním složky, byla i pipeline navržena jako plně dynamická. Provádí následující operace:
 1. Prochází podadresáře v adresáři `./runners/` a pro každý nalezený jazyk (např. cs, cpp, java) přečte jeho Dockerfile a pomocí proudového editoru `sed` vyparsuje verzi základního obrazu (řádek `FROM ... AS final`).
 2. Dynamicky sestaví tab obrazu (např. `test-runner-cs:8.0`) a spustí `docker build`.
 3. Seznam nově vytvořených obrazů předá další úloze, která zajistí jejich hromadné odeslání do registru. Tento přístup snižuje režii spojenou s údržbou systému. Při nasazení podpory pro nový programovací jazyk není potřeba upravovat CI/CD pipeline, protože skript novou složku najde a zpracuje autonomně.

5 Implementace

Tato kapitola popisuje implementaci navrženého systému pro automatizované vyhodnocování programátorských úloh. Zaměřuje se především na průběh zpracování jednotlivých požadavků, komunikaci mezi komponentami systému a způsob zajištění izolovaného spuštění uživatelského kódu. Součástí kapitoly je také popis backendové aplikace, kontejnerových runnerů a způsobu komunikace s externími službami.

5.1 TO DO: Workflow systému

UML :(

Endpoint POST url -> aplikace zkontroluje url (jestli githubí, jestli se na ni dostane) -> přes Github API získá jazyk -> zkontroluje jestli jazyk je podporován -> uloží se záznam o assignment do databáze -> PodmanManager udělá kontejner pro url s jazykem, spustí -> validace ->

Case: spuštěný kód je ok! -> return přes RabbitMQ -> InQuest sní ->

Case: kontejner je v cyklu/nepošle zprávu pro RabbitMQ -> InQuest polluje kontejnery a kontroluje jak dlouho běží. Jak přijde limit, smaže kontejner

Final: -> uloží data pod Result záznam

Endpoint GET all -> get

Endpoint PUT -> znovu zvaliduje záznam

And so on

Jsou vlastně následující sekce potřeba? :skull:

5.2 Průběh zpracování požadavku

Vyhodnocení programátorské úlohy začíná přijetím HTTP požadavku na REST API backendové aplikace. Klient předává adresu GitHub repozitáře obsahujícího řešení zadané úlohy. Backend nejprve ověří, že se jedná o platnou adresu GitHub repozitáře a že je repozitář dostupný.

Po úspěšné validaci backend prostřednictvím GitHub API získá informace o repozitáři, především hlavní programovací jazyk projektu. Pokud není daný jazyk podporován, je požadavek ukončen a klient obdrží odpovídající chybovou odpověď. V opačném případě je v databázi vytvořen nový záznam typu *Assignment*, který reprezentuje požadavek na vyhodnocení.

Na základě zjištěného programovacího jazyka backend zvolí odpovídající kontejnerový obraz a prostřednictvím komponenty PodmanManager vytvoří nový kontejner. Do kontejneru jsou předány informace potřebné pro zpracování úlohy, zejména adresa GitHub repozitáře a přístupové údaje uložené pomocí mechanismu Podman Secrets.

Po spuštění kontejneru runner naklonuje zadaný repozitář, ověří jeho strukturu a provede sestavení projektu. Pokud je sestavení úspěšné, aplikace je spuštěna a jsou zaznamenány informace o průběhu sestavení i spuštění. Výsledky

jsou následně serializovány a odeslány prostřednictvím RabbitMQ zpět backendové aplikaci.

Backend po přijetí zprávy uloží získané informace do databáze jako záznam typu *Result*. Současně zajistí odstranění vytvořeného kontejneru, čímž dojde k uvolnění využitých systémových prostředků.

V některých případech nemusí být vyhodnocení dokončeno korektně. Může se jednat například o nekonečnou smyčku programu, pád aplikace nebo neočekávané ukončení runneru bez odeslání výsledků. Backend proto pravidelně kontroluje běžící kontejnery a jejich dobu běhu. Pokud kontejner překročí nastavený časový limit, je ukončen a odstraněn. O výsledku vyhodnocení je následně vytvořen odpovídající záznam v databázi.

5.3 TO DO: Příjetí požadavku

POST endpoint
validace URL
Github API
uložení assignmentu

5.4 TO DO: Izolovaná exekuční prostředí (Runnery)

Každý podporovaný programovací jazyk disponuje vlastním dedikovaným kontejnerovým obrazem (tzv. *runnerem*).

5.4.1 TO DO: Spuštění izolovaného prostředí

PodmanManager
vytvoření kontejneru
podman secrets
Backendová aplikace komunikuje s Podmanem prostřednictvím jeho vzdáleného REST API a rozhraní *PodmanManager*. Při přijetí nového požadavku na vyhodnocení backend dynamicky:

1. Identifikuje požadovaný jazyk a zvolí příslušný kontejnerový obraz.
2. Vytvoří a spustí nový kontejner.
3. Předá kontejneru parametry (např. URL adresa testovaného repozitáře).
4. Po dokončení vyhodnocení a odeslání výsledků kontejner bezpečně odstraní.

5.4.2 BASH skripty a úprava prostředí

Vnitřní logika runneru je řízena spouštěcím BASH skriptem (např. `start.sh`). Skript po spuštění kontejneru klonuje zadaný GitHub repozitář, přeloží zdrojové kódy a spustí připravenou sadu testů.

Pro zajištění dynamické konfigurace a přípravu obrazů byly použity standardní POSIX nástroje:

- **xmlstarlet**: zajišťuje automatickou modifikaci .NET projektových souborů (`.csproj/Directory.Build.props`), kam dynamicky zapisuje informace o verzi sestavení získané z CI/CD pipeline.
- **sed**: slouží k proudovému parsování a úpravám textových souborů (např. extrakce přesné verze runtime prostředí přímo ze souborů `Dockerfile`).

Díky modularitě BASH skriptů je systém snadno rozšiřitelný. Přidání podpory pro nový programovací jazyk vyžaduje pouze vytvoření nové složky s příslušným `Dockerfile` a spouštěcím skriptem.

5.5 TO DO: Zpracování výsledku

RabbitMq timeout polling Result

5.6 TO DO: Ukázky použití

Swagger POST GET PUT

6 TO FINISH: Budoucí rozšíření

Navržený systém byl vytvořen s důrazem na modularitu a snadnou rozšiřitelnost. Současná implementace splňuje stanovené požadavky, nicméně architektura umožňuje další rozvoj, který může zvýšit kvalitu systému, usnadnit jeho údržbu a rozšířit možnosti jeho využití.

- **TO DO: Možnost nastavit výsledek** Nastavit výsledek, podle kterého má být úkolový výsledek kontorlován
- **Automatizované testování v CI/CD** Současná CI/CD pipeline zajišťuje sestavení aplikace a vytvoření kontejnerových obrazů. V budoucnu by bylo vhodné pipeline rozšířit o fázi, která zajišťuje automatické spouštění testů před samotným sestavením a nasazením aplikace. Díky tomu by bylo možné zachytit případné chyby během vývojového procesu a zabránit nasazení nefunkčních verzí systému.
- **Podpora dalších programovacích jazyků** Architektura systému byla navržena tak, aby bylo možné jednoduše přidávat podporu dalších programovacích jazyků. Rozšíření spočívá především ve vytvoření nového runneru obsahujícího odpovídající kontejnerový obraz a skript definující proces sestavení a spuštění projektu. Díky dynamickému zpracování runnerů v CI/CD pipeline není nutné upravovat samotnou backendovou aplikaci. V současné verzi aplikace podporuje jazyky C#, C++, Java, Python, Rust, JavaScript a TypeScript.
- **Rozšíření REST API a správa dat** Stávající REST rozhraní se zaměřuje na základní životní cyklus jednotlivých požadavků. Pro usnadnění správy a integraci s rozsáhlejšími systémy by bylo v budoucnosti vhodné API rozšířit o nové endpointy:
 - **Hromadné zpracování požadavků (POST /api/batch):** Umožnilo by odeslat pole odkazů na repozitáře v jediném HTTP požadovaném rozhraní. Systém by následně hromadně vytvořil frontu úloh pro spuštění v runneru, což by zjednodušilo obsluhu pro větší počet úloh naráz.
 - **Správa a promazávání dat (DELETE /api/all):** Administrátorský endpoint určený k hromadnému mazání historických záznamů a výsledků (případně s možností filtrů podle data či stavu). To by usnadnilo údržbu databáze a uvolňování místa na disku.

7 TO DELETE: Sazba částí dokumentu

7.1 Sazba úvodní strany či obsahu

Vysázení všech podstatných částí úvodu práce obstará makro `\maketitle`. Pro správné vysázení všech částí a meta-informací je potřeba použít makra `\title`, `\author` a další. Jejich přehled lze najít ve zdrojovém souboru tohoto dokumentu. V případě použití **pdf** výstupu se generuje i dodatečná hlavička souboru s meta-informacemi jako je autor dokumentu, název práce či dalšími.

7.2 Závěry

Závěr práce by se měl poskytnout jak v původním jazyce práce, tak v jazyce anglickém. Pro sazbu závěru jsou k dispozici příslušná makra. Berte na vědomí, že , se aktivuje plně anglická sazba se všemi konvencemi. Tedy je třeba používat anglické uvozovky a další správné typografické prvky.

```
1 % Tiskne český závěr práce.
2 \begin{kiconclusions}
3 Závěr práce v \uv{českém} jazyce.
4 \end{kiconclusions}
5
6 % Tiskne anglický závěr práce.
7 \begin{kiconclusions}[english]
8 Thesis conclusions written in \uv{English}.
9 \end{kiconclusions}
```

Zdrojový kód 1: Sazba závěrů

7.3 Sazba literatury

Pro sazbu literatury má uživatel dvě možnosti. Může použít služby balíků `BIBLATEX`, který je pro **kistyles** zapnutý, či lze použít manuální sazbu bibliografie.

7.4 Drobná makra

Základní styl definuje hned několik maker pro usnadnění práce. Například makro `\buno` vysází řetězec „bez újmy na obecnosti“. Je k dispozici i verze s prvním velkým písmenem, `\Buno`.

Je rovněž možno přidávat položky do seznamu zkratk. K tomu slouží makro `\newacronym`, které lze použít například jednoduše jako `\newacronym{UPOL}{UPOL}{\kitextunivcz}`. Na danou zkratku se pak lze odkazovat jednoduše, `\gls{UPOL}`.

Sazba uvozovek respektuje nastavení částí dokumentu, a proto se doporučuje používat makro `\uv`. V anglické závěru práce toto platí taky, viz tato PDF ukázka.

Styl podporuje sazbu odstavců v tabulkách, více obsahuje tabulka [1](#).
K dispozici jsou také makra pro sazbu C# (`\csharp`) či C++ (`\cpp`).

7.5 Sazba rejstříku

Sazba rejstříku sestává z několika kroků:

1. Je třeba přes volbu `index=true` rejstříkování povolit.
2. Použitím makra `\index` rejstříkovat vybrané pojmy.
3. Kompilovat s použitím utility `makeindex`. Pro specifiky tohoto kroku si stačí prohlédnout soubor `Makefile`.

Makro `\index` je redefinováno tak, že sází klikací odkaz na výraz v rejstříku. Je doporučeno jej použít ihned za výrazem [{1}](#).

Omezení redefinovaného makra `\index`: klikací odkaz nefunguje, pokud použijete konstrukci `\index{výraz|makro}` (resp. `\index{výraz|(makro)}`), např. `\index{výraz|textit}`.

Rejstřík lze vysázet pomocí makra `\printindex`.

7.6 Sazba zdrojových kódů

Styl nabízí dva způsoby sazby zdrojových kódů:

1. Sazbu řádkových kódů, například `background-color: white;`. K tomu slouží makro formátu `\kiinlinecode{jazyk}{separátor}{kód}`. Za separátor je vhodné volit jakýkoliv znak, který se nevyskytuje v samotném sázeném zdrojovém kódu. Za jazyk je nutno dosadit jeden z těchto: C, TeX, PHP, HTML, Lisp, SQL, TeX, Python, Java, TutorialD, text, csharp, cpp, JavaScript, CSS.
2. Sazbu zdrojových kódů do separátních prostředí. Takto vytištěný kód se objeví v seznamu zdrojových kódů. Ukázka například zdrojový kód [2](#). Ukázku sazby naleznete ve zdrojovém kódu tohoto dokumentu.

```
1 int main("cs acsa") // komentar
2 int main("cs acsa") // komentar
3 int main("cs acsa") // komentar
4 int main("cs acsa") // komentar
5 int main("cs acsa") // komentar
```

Zdrojový kód 2: C++

Tabulka 1: Odstavce v tabulkách

Donec et nisl id sapien blandit mattis. Aenean dictum odio sit amet risus. Morbi purus. Nulla a est sit amet purus venenatis iaculis. Vivamus viverra purus vel magna. Donec in justo sed odio malesuada dapibus. Nunc ultrices aliquam nunc. Vivamus facilisis pellentesque velit. Nulla nunc velit, vulputate dapibus, vulputate id, mattis ac, justo. Nam mattis elit dapibus purus. Quisque enim risus, congue non, elementum ut, mattis quis, sem. Quisque elit.	Etiam suscipit aliquam arcu. Aliquam sit amet est ac purus bibendum congue. Sed in eros. Morbi non orci. Pellentesque mattis lacinia elit. Fusce molestie velit in ligula. Nullam et orci vitae nibh vulputate auctor. Aliquam eget purus. Nulla auctor wisi sed ipsum. Morbi porttitor tellus ac enim. Fusce ornare. Proin ipsum enim, tincidunt in, ornare venenatis, molestie a, augue. Donec vel pede in lacus sagittis porta. Sed hendrerit ipsum quis nisl. Suspendisse quis massa ac nibh pretium cursus. Sed sodales. Nam eu neque quis pede dignissim ornare. Maecenas eu purus ac urna tincidunt congue.	Etiam pede massa, dapibus vitae, rhoncus in, placerat posuere, odio. Vestibulum luctus commodo lacus. Morbi lacus dui, tempor sed, euismod eget, condimentum at, tortor. Phasellus aliquet odio ac lacus tempor faucibus. Praesent sed sem. Praesent iaculis. Cras rhoncus tellus sed justo ullamcorper sagittis. Donec quis orci. Sed ut tortor quis tellus euismod tincidunt. Suspendisse congue nisl eu elit. Aliquam tortor diam, tempus id, tristique eget, sodales vel, nulla. Praesent tellus mi, condimentum sed, viverra at, consectetur quis, lectus. In auctor vehicula orci. Sed pede sapien, euismod in, suscipit in, pharetra placerat, metus. Vivamus commodo dui non odio. Donec et felis.
---	--	--

```
1 new object() // komentar
```

Zdrojový kód 3: JS

```
1 public static int main("cs aca") // komentar
```

Zdrojový kód 4: C#

```
1 SELECT * FROM table_1; /* komentar */
```

Zdrojový kód 5: SQL

```
1 table_1 AND table_2;
```

Zdrojový kód 6: TutorialD

Závěr

TODO: Závěr práce v „českém“ jazyce.

Conclusions

TODO: Thesis conclusions in “English”.

A TODO: Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

README.*

Textový soubor (s příponou např. .txt) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,
- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwarového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovoluují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.*.

Literatura

- [1] Microsoft. *.NET / Free. Cross-platform. Open source* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://dotnet.microsoft.com/en-us/>.
- [2] Getachew, Samuel. *Introduction to ASP.NET Core Web API: A Complete Guide for Developers* [online]. 2024 [cit. 2026-6-22]. Dostupný z: <https://medium.com/@solomongetachew112/introduction-to-asp-net-core-web-api-a-complete-guide-for-developers-347df16b8ab6>.
- [3] Microsoft. *Migrations Overview* [online]. 2024 [cit. 2026-6-22]. Dostupný z: <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/>.
- [4] Code, Bob. *Entity Framework Core (Code-First): Introduction, Best Practices, Repository Pattern & Clean Architecture* [online]. 2024 [cit. 2026-6-22]. Dostupný z: <https://medium.com/@codebob75/entity-framework-core-code-first-introduction-best-practices-repository-pattern-clean-22b6152bcb81>.
- [5] SmartBear Software. *API Documentation & Design Tools / Swagger* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://swagger.io/>.
- [6] Pranaya, Rout. *AutoMapper in C#* [online]. 2023 [cit. 2026-6-22]. Dostupný z: <https://dotnettutorials.net/lesson/automapper-in-c-sharp/>.
- [7] Serilog Contributors. *Serilog* [online]. 2026 [cit. 2026-6-26]. Dostupný z: <https://serilog.net/>.
- [8] Red Hat, Inc. *What is a REST API?* [online]. 2020 [cit. 2026-6-22]. Dostupný z: <https://www.redhat.com/en/topics/api/what-is-a-rest-api>.
- [9] Podman Container Tools. *Podman* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://podman.io/>.
- [10] Red Hat, Inc. *What is Podman?* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://www.redhat.com/en/topics/containers/what-is-podman>.
- [11] Preston, Dan. *Keep your secrets secret with the new Podman secrets command* [online]. 2021 [cit. 2026-6-27]. Dostupný z: <https://www.redhat.com/en/blog/new-podman-secrets-command>.
- [12] Jain, Urvashi. *Podman REST API* [online]. 2022 [cit. 2026-6-27]. Dostupný z: <https://www.redhat.com/en/blog/podman-rest-api>.
- [13] Broadcom Inc. *Messaging that just works — RabbitMQ* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://www.rabbitmq.com/>.
- [14] GitLab Inc. *GitLab CI/CD documentation* [online]. 2026 [cit. 2026-6-26]. Dostupný z: <https://docs.gitlab.com/ci/>.

- [15] Red Hat, Inc. *What is YAML?* [online]. 2023 [cit. 2026-6-26]. Dostupný z: <https://www.redhat.com/en/topics/automation/what-is-yaml>.
- [16] Sanil, Zayna. *Bash Scripting Tutorial – Linux Shell Script and Command Line for Beginners* [online]. 2023 [cit. 2026-6-26]. Dostupný z: <https://www.freecodecamp.org/news/bash-scripting-tutorial-linux-shell-script-and-command-line-for-beginners/>.
- [17] Saluja, Ashish. *Getting Started with GitHub REST API* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://www.loginradius.com/blog/engineering/github-api>.
- [18] Martin, Robert C. *The Clean Architecture* [online]. 2012 [cit. 2026-6-22]. Dostupný z: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>.